Basic usage + client
e.g. simple usage

API

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE DeriveAnyClass #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE TypeOperators #-}
{-# OPTIONS_GHC -Wno-missing-export-lists #-}

module ServantAPI
  ( User,
    UserId,
    BaseAPI,
    UserAPI,
    API,
    TargetUserAPI,
    api,
    mainHandler,
  )
where

import Control.Monad.IO.Class
import Data.Aeson
import GHC.Generics
import Servant

-- Data
data User where
  User :: {userId :: UserId, name :: String, email :: String} ->
User
  deriving stock (Generic, Show, Eq)
  deriving anyclass (FromJSON, ToJSON)

type UserId = String

api :: Proxy API
api = Proxy


-- API
type API =
  "api"
    :> Summary "Main API"
    :> Description
         "This is a sample API\
         \5 endpoints \
         \ GET      /api/user \
         \ POST     /api/user \
         \ GET      /api/user/:uid \
         \ POST     /api/user/:uid \
```

```
        \ DELETE   /api/user/:uid "
    :> ( BaseAPI
           :<|> UserAPI
       )

type UserAPI =
  "user"
    :> Summary "User API"
    :> ( Get '[JSON] [User]
           :<|> ReqBody '[JSON] User :> Post '[JSON] ()
       )

type CaptureUID = Capture "uid" UserId
type TargetUserAPI =
  ( CaptureUID :> Get '[JSON] User
      :<|> CaptureUID :> ReqBody '[JSON] User :> Post '[JSON] ()
      :<|> CaptureUID :> Delete '[JSON] ()
  )


-- Handlers
mainHandler :: Server API
mainHandler =
  baseAPIHandler
    :<|> userAPIHandler
    :<|> targetUserAPIHandler

baseAPIHandler :: Server BaseAPI
baseAPIHandler = return "Hello"

userAPIHandler :: Server UserAPI
userAPIHandler =
  ( do
      liftIO $ putStrLn "GET user"
      return []
  )
    :<|> createUser

targetUserAPIHandler :: Server TargetUserAPI
targetUserAPIHandler =
  ( \uid -> do
      liftIO $ putStrLn $ "GET uid:" <> uid
      return $ User "123" "Yui" "test@test.com"
  )
    :<|> updateTargetUser
    :<|> deleteTargetUser

createUser :: User -> Handler ()
createUser user = do
  liftIO $ putStrLn $ "POST user:" <> show user
  return ()

updateTargetUser :: String -> User -> Handler ()
```

```
updateTargetUser uid user = do
  liftIO $ putStrLn $ "POST uid:" <> uid <> " user:" <> show user
  return ()

deleteTargetUser :: String -> Handler ()
deleteTargetUser uid = do
  liftIO $ putStrLn $ "DELETE uid:" <> uid
  throwError err403
```

Client

```
{-# LANGUAGE DataKinds #-}

module ServantClient
  ( runClient,
    getTargetUser,
    updateTargetUser,
    deleteTartgetUser,
    getUsers,
    createUser,
    getBase,
  )
where

import Network.HTTP.Client hiding (Proxy)
import Servant
import Servant.Client
import ServantAPI

runClient :: BaseUrl -> ClientM a -> IO (Either ClientError a)
runClient url cl = do
  manager <- newManager defaultManagerSettings
  runClientM cl (mkClientEnv manager url)

getTargetUser :: UserId -> ClientM User
updateTargetUser :: UserId -> User -> ClientM ()
deleteTartgetUser :: UserId -> ClientM ()
getTargetUser :<|> updateTargetUser :<|> deleteTartgetUser = client
(Proxy @TargetUserAPI)

getUsers :: ClientM [User]
createUser :: User -> ClientM ()
getUsers :<|> createUser = client (Proxy @UserAPI)

getBase :: ClientM String
getBase = client (Proxy @BaseAPI)
```

Main

```
{-# LANGUAGE OverloadedStrings #-}

import ServantAPI
```

```
import Data.ByteString qualified as BS
import Data.Function ((&))
import Network.Wai
import Network.Wai.Handler.Warp
import Servant

-- Error handlers
errorFromatter :: ErrorFormatters
errorFromatter =
  defaultErrorFormatters
    { notFoundErrorFormatter = \req ->
        err404 {errBody = "404 Not found:" <> BS.fromStrict
(rawPathInfo req)}
    }

-- Server settings
serverSettings :: Settings
serverSettings =
  defaultSettings
    & setPort 1234
    & setHost "localhost"

start :: IO ()
start =
  serveWithContext api (errorFromatter :. EmptyContext)
mainHandler
    & runSettings serverSettings
```

e.g. Start server

```
ghci> start
```

e.g. call Server

```
$ curl -XGET localhost:1234/api                 # Hello
$ curl -XGET localhost:1234/abc                 # 404 Not Found: /
abc
```

Usage: define custom request/response body type
Using content types: https://docs.servant.dev/en/stable/tutorial/Server.html#using-content-types-with-your-data-types
Need to define

Accept class	define HTTP **Content-Type** header
MimeRender class	render data -> HTTP response body
MimeParse class	parse data in HTTP request body

e.g. JSON as raw binary data

```
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE TypeOperators #-}
```

```
import Servant
import Network.HTTP.Media hiding (Accept)
import Data.ByteString.Char8 qualified as B
import Control.Monad.IO.Class

data RawJSON

instance Accept RawJSON where
  contentType _ = "application" // "json" /: ("charset", "utf-8")

instance MimeRender RawJSON B.ByteString where
  mimeRender _ = B.fromStrict

instance MimeUnrender RawJSON B.ByteString where
  mimeUnrender _ = Right . B.toStrict
```

Use in API

```
type API =
  Get '[RawJSON] B.ByteString                                  --
HTTP Response body
  :<|> ReqBody '[RawJSON] B.ByteString :> Post '[JSON] ()    --
HTTP Request body

handler :: Server API
handler = liftIO (B.readFile "data.json")
  :<|> (liftIO . B.putStrLn )
```

Usage: define custom parameter type (query param, header, capture)
Web.HttpApiData: https://hackage.haskell.org/package/http-api-data
Define

ToHttpApiData class	render e.g. custom type -> query param
FromHttpApiData class	parse parameter e.g. query param -> custom type

```
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE TypeOperators #-}

import Servant
import Data.ByteString.Char8 qualified as B

data MyParam = A | B

instance ToHttpApiData MyParam where
  toQueryParam A = "a"
  toQueryParam B = "b"

instance FromHttpApiData MyParam where
  parseQueryParam "A" = Right A
```

```
  parseQueryParam "B" = Right B
  parseQueryParam t = Left $ "Failed to parse " <> t
```

Use in API

```
type API3 =
  QueryParam "myvalue" MyParam :> Get '[JSON] ()        -- as query
param
  :<|> Capture "name" MyParam :> Get '[JSON] ()         -- as
capture
  :<|> Header "X-name" MyParam :> Get '[JSON] ()        -- as
header value
```

Usage: augment Handler with transforms
Using a custom monad: https://docs.servant.dev/en/stable/cookbook/using-custom-monad/UsingCustomMonad.html
Use **ServerT** + **hoistServer** to convert ServerT to Server

e.g. add state + logger

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE GeneralizedNewtypeDeriving #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE TypeOperators #-}

import Control.Monad.Catch hiding (Handler)
import Control.Monad.Error (MonadError)
import Control.Monad.Logger
import Control.Monad.State
import Servant
import Servant.HTML.Blaze
import Data.Function ((&))

newtype MyHandler a = MyHandler (LoggingT (StateT String Handler)
a)        --
  deriving newtype
    ( Functor,
      Applicative,
      Monad,
      MonadThrow,
      MonadCatch,
      MonadLogger,
      MonadError ServerError
    )

type MyAPI = Get '[JSON] String :<|> "number" :> Get '[JSON] Int

myAPI :: Proxy MyAPI
myAPI = Proxy
```

```
-- step: define with transforms
myAPIServer :: ServerT MyAPI MyHandler

myAPIServer =
  ( do
      logDebugN "root called"
      pure "OK"
  )
    :<|> ( do
             logDebugN "number called"
             pure 1
         )

-- step: convert transforms to Server MyAPI
mainServer :: Server MyAPI -- Server MyAPI = ServerT MyAPI Handler
mainServer = hoistServer myAPI f myAPIServer
  where
    f :: MyHandler a -> Handler a
         -- converter
    f (MyHandler h) = runStdoutLoggingT h
      & flip evalStateT ""
```

Usage: add custom error handlers
Customizing errors: https://docs.servant.dev/en/stable/cookbook/custom-errors/CustomErrors.html
Create a custom **ErrorFormatter**
Add error

```
{-# LANGUAGE OverloadedStrings #-}

import ServantAPI
import Data.ByteString qualified as BS
import Network.Wai
import Network.Wai.Handler.Warp
import Servant

-- API
type API = ...
mainHandler :: Server API
mainHandler = ...


-- Error handlers
errorFromatter :: ErrorFormatters
errorFromatter =
  defaultErrorFormatters
    { notFoundErrorFormatter = \req ->
                  -- req :: Network.Wai.Request
        err404 {errBody = "404 Not found:" <> BS.fromStrict
(rawPathInfo req)}
    }
```

```
server :: Application
server = serveWithContext (Proxy @API) (errorFromatter :.
EmptyContext) mainHandler
```

Usage: access **Request** in a Server
Just **explicitly** create a Server for each request.
Application is just:

type Application = Request -> (Response -> IO ResponseReceived) -> IO ResponseReceived

e.g.

```
createServer :: Request -> Server API
createServer req = ...

createApplication :: Application
createApplication req =
  let app = serve (Proxy @API) (createServer req)
  in app req
```

Testing with Hspec
Main points:
- use **ClientM** to call the API in tests.
  pass **ClientEnv** to the tests
- store the server **shutdown handler** in a **shared memory** variable e.g. Control.Concurrent.STM.TVar
- start the server on a different **thread**, and stop the server with the shutdown handler after tests

e.g.

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE LambdaCase #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE TypeOperators #-}

import Control.Concurrent
import Control.Concurrent.STM
import Control.Monad
import Control.Monad.IO.Class
import Data.Function
import GHC.Exts (IsString)
import Network.HTTP.Client (defaultManagerSettings, newManager)
import Network.HTTP.Types (status404)
import Network.Wai.Handler.Warp
import Servant
import Servant.Client
import Test.Hspec
```

```
testHost :: (IsString s) => s
testHost = "localhost"

testPort :: Port
testPort = 12345

type ShutdownHandler = TVar (IO ())

requestShutdown :: (MonadIO m) => ShutdownHandler -> m ()
requestShutdown = liftIO . join . readTVarIO

newShutdownHandler :: (MonadIO m) => m ShutdownHandler
newShutdownHandler = liftIO $ newTVarIO (pure ())

startServer :: (MonadIO m) => ShutdownHandler -> m ()
startServer shutdown =
  liftIO
    $ runSettings
      (testServerSettings shutdown)
      (serve (Proxy @API) handler)

testServerSettings :: ShutdownHandler -> Settings
testServerSettings shutdown =
  defaultSettings
    & setPort testPort
    & setHost testHost
    & setInstallShutdownHandler (atomically . writeTVar shutdown)
                -- install shutdown handler
```

Server

```
type API =
  "api"
    :> ( "a"
           :> QueryParam' '[Required] "name" String
           :> Get '[JSON] String
           :<|> ReqBody '[JSON] Int :> Post '[JSON] Int
       )

handler :: Server API
handler =
  (\name -> return $ "GET:" <> name)
    :<|> \num -> return (1 + num)
```

Client

```
getCall :: String -> ClientM String
postCall :: Int -> ClientM Int
getCall :<|> postCall = client (Proxy @API)

type NonExistentEndpoint = "api" :> "nonexistent" :> Get '[JSON]
```

```
()               -- Non-existent API client for testing error
handling

getNonExistent :: ClientM ()
getNonExistent = client (Proxy @NonExistentEndpoint)
```

Tests

```
oneSecond :: Int
oneSecond = 1000 * 1000

testUrl :: BaseUrl
testUrl = BaseUrl Http testHost testPort ""

spec :: Spec
spec =
  ( \runTest -> do
      -- Step: create settings
      shutdownHandler <- newShutdownHandler
      manager <- newManager defaultManagerSettings
      let clientEnv = mkClientEnv manager testUrl

      -- Step: Start server on a different thread
      _ <- forkIO $ do
        putStrLn $ "Starting server " <> testHost <> ":" <> show
testPort
        startServer shutdownHandler

      -- Note: wait a bit to allow server to start
      threadDelay oneSecond

      -- Step: run test
      runTest clientEnv

      -- Step: cleanup
      requestShutdown shutdownHandler
  )
    `aroundAll` baseSpec

baseSpec :: SpecWith ClientEnv
baseSpec = describe "API test" $ do
  let expectSuccess :: (HasCallStack) => ClientM a -> ClientEnv ->
(a -> Expectation) -> Expectation
      expectSuccess call env onSuccess = do
        result <- runClientM call env
        case result of
          Right value -> onSuccess value
          Left e -> expectationFailure $ "Unexepcted error:" <>
show e

      expectFailure :: (HasCallStack, Show a) => ClientM a ->
```

```
ClientEnv -> (ClientError -> Expectation) -> Expectation
      expectFailure call env onFailure = do
        result <- runClientM call env
        case result of
          Right value -> expectationFailure $ "" <> show value
          Left f -> onFailure f

  it "GET /api/a/?name=<value>" $ \env -> do
    expectSuccess (getCall "abc") env (`shouldBe` "GET:abc")

  it "POST /api/a" $ \env -> do
    expectSuccess (postCall 1) env (`shouldBe` 2)

  it "[ERROR] non-existent endpoint" $ \env -> do
    expectFailure getNonExistent env $ \case
      (FailureResponse _ res) -> responseStatusCode res `shouldBe`
status404
      e -> expectationFailure $ "Unexpected error:" <> show e
```

==========

HasServer

Defining my own **servers** i.e. HasServer instances
Need to implement
- **HasServer** instances
- **HasClient** instances

e.g. setup

```
{-# LANGUAGE ConstraintKinds #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE InstanceSigs #-}
{-# LANGUAGE LambdaCase #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE RankNTypes #-}
{-# LANGUAGE RecordWildCards #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TemplateHaskell #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE TypeOperators #-}
{-# LANGUAGE UndecidableInstances #-}
{-# OPTIONS_GHC -Wno-missing-export-lists #-}
{-# OPTIONS_GHC -Wno-orphans #-}

import Control.Concurrent
import Control.Concurrent.STM
```

```
import Control.Monad
import Control.Monad.IO.Class
import Data.ByteString.Char8 qualified as B
import Data.Foldable
import Data.Function
import Data.Kind
import Data.Map qualified as M
import Data.Singletons.TH
import Data.Text (Text)
import Data.Text qualified as T
import Data.Text.Encoding
import Data.Text.IO qualified as T
import Data.Typeable
import GHC.TypeLits
import Network.HTTP.Client (defaultManagerSettings, newManager)
import Network.HTTP.Media (renderHeader)
import Network.HTTP.Types
import Network.Wai
import Network.Wai.Handler.Warp
import Servant
import Servant.API.Status
import Servant.Client hiding (Response)
import Servant.Client.Core (RunClient, appendToPath,
appendToQueryString)
import Servant.Client.Core qualified as C
import Servant.Server.Internal.Delayed
import Servant.Server.Internal.DelayedIO
import Servant.Server.Internal.RouteResult
import Servant.Server.Internal.Router
import Test.Hspec
import Control.Monad.Reader (MonadReader(..))

data MyVerbName = MyGETName | MyPOSTName

-- Singleton for converting type to value
genSingletons [''MyVerbName]

instance {-# OVERLAPPABLE #-} (Show a) => MimeRender PlainText a
where
  mimeRender _ = B.fromStrict . encodeUtf8 . T.pack . show

instance {-# OVERLAPPABLE #-} (Read a) => MimeUnrender PlainText a
where
  mimeUnrender _ value = case (reads @a) text of
    [(v, _)] -> Right v
    _ -> Left $ "Failed to decode: " <> text
   where
    text = T.unpack $ decodeUtf8 $ B.toStrict value
```

Empty server i.e. a server that always returns an error

HasServer MyEmpty
HasClient m MyEmpty

```
data MyEmpty

emptyHandler :: (Monad m) => ServerT MyEmpty m
emptyHandler = return ()

instance HasServer MyEmpty ctx where
  type ServerT MyEmpty m = m ()
  route _ _ delayedRouter =
    StaticRouter

      M.empty
      [ \env req k ->
         runAction delayedRouter env req k
           (\_ -> Fail $ err500{errReasonPhrase = "EMPTY SERVER"})
      ]
  hoistServerWithContext _ _ f = f

instance (RunClient m) => HasClient m MyEmpty where
  type Client m MyEmpty = m ()
  clientWithRoute _ _ req = void $ C.runRequestAcceptStatus Nothing
req
  hoistClientMonad _ _ f = f
```

Path
HasServer (MyPath path :> api)
HasClient m (MyPath path :> api)

```
data MyPath (path :: Symbol)

instance (HasServer api ctx, KnownSymbol path) => HasServer (MyPath
path :> api) ctx where
  type ServerT (MyPath path :> api) m = ServerT api m

  route ::
    Proxy (MyPath path :> api) ->
    Context ctx ->
    Delayed env (Server api) ->
    Router env
  route _ ctx delayedRouter = StaticRouter (M.fromList [(T.pack
pathName, router')]) []
   where
    pathName = symbolVal @path Proxy
    router' = route (Proxy @api) ctx delayedRouter
  hoistServerWithContext _ = hoistServerWithContext (Proxy @api)

instance (KnownSymbol path, HasClient m api) => HasClient m (MyPath
path :> api) where
  type Client m (MyPath path :> api) = Client m api
  clientWithRoute _ _ req = clientWithRoute (Proxy @m) (Proxy @api)
```

```
$ appendToPath (T.pack $ symbolVal (Proxy @path)) req
  hoistClientMonad _ _ = hoistClientMonad (Proxy @m) (Proxy @api)
```

Capture
HasServer (Capture name a :> api)
HasClient m (Capture name a :> api)

```
data MyCapture (name :: Symbol) (a :: Type)

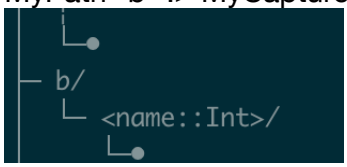
instance (HasServer api ctx, FromHttpApiData a, KnownSymbol name,
Typeable a) => HasServer (MyCapture name a :> api) ctx where
  type ServerT (MyCapture name a :> api) m = a -> ServerT api m

  route ::
    forall env.
    Proxy (MyCapture name a :> api) ->
    Context ctx ->
    Delayed env (a -> Server api) ->
    Router env
  route _ ctx delayedRouter = CaptureRouter [captureHint] $ route
(Proxy @api) ctx delayedRouter'
   where
    captureHint = CaptureHint (T.pack $ symbolVal (Proxy @name))
(typeRep (Proxy @a))                                  -- used to show
capture in layout
    routeNext :: Delayed env (a -> Server api) -> Delayed (Text,
env) (Server api)
    routeNext Delayed{..} =
      Delayed
        { capturesD = \(text, env) ->
             (,) <$> parseCapture text <*> capturesD env
                                                          -- step 1:
parse capture
        , serverD = \(a, caps) params headers auth body req ->
             ($ a) <$> serverD caps params headers auth body req
                                                          -- step 2:
compute RouteResult
        , ..
        }
    delayedRouter' = routeNext delayedRouter
    delayedRouter'' = addCapture delayedRouter parseCapture
                                                          -- same
    parseCapture text = case parseUrlPiece text of
      Right a -> return a
      Left err -> delayedFail $ err400{errReasonPhrase = T.unpack
("Failed to parse capture /:" <> text <> "\n" <> err)}
  hoistServerWithContext _ ctx f serverHandler =
hoistServerWithContext (Proxy @api) ctx f . serverHandler

instance (ToHttpApiData a, HasClient m api) => HasClient m
(MyCapture name a :> api) where
  type Client m (MyCapture name a :> api) = a -> Client m api
```

```
  clientWithRoute _ _ req value = clientWithRoute (Proxy @m) (Proxy
@api) $
    appendToPath (toUrlPiece value) req
  hoistClientMonad _ _ f = (hoistClientMonad (Proxy @m) (Proxy
@api) f <$>)
```

e.g.
MyPath "b" :> MyCapture "name" Int



QueryParam
HasServer (MyQueryParam name a :> api)
HasClient m (MyQueryParam name a :> api)

```
data MyQueryParam (name :: Symbol) (a :: Type)

instance (HasServer api ctx, KnownSymbol name, FromHttpApiData a)
=> HasServer (MyQueryParam name a :> api) ctx where
  type ServerT (MyQueryParam name a :> api) m = a -> ServerT api m

  route ::
    Proxy (MyQueryParam name a :> api) ->
    Context ctx ->
    Delayed env (a -> Server api) ->
    Router env
  route _ ctx delayedRouter = route (Proxy @api) ctx delayedRouter'
   where
    name = symbolVal (Proxy @name)
    routeNext :: Delayed env (a -> Server api) -> Delayed env
(Server api)
    routeNext Delayed{..} =
      Delayed
        { paramsD =
            (,) <$> (ask >>= parseQueryParamFromRequest @a name)
<*> paramsD                                          -- step 1:
parse
        , serverD = \cap (a, params) headers auth body req ->
            ($ a) <$> serverD cap params headers auth body req
                                                     -- step 2:
compute RouteResult
        , ..
        }
    delayedRouter' = routeNext delayedRouter
    delayedRouter'' = addParameterCheck delayedRouter (ask >>=
parseQueryParamFromRequest @a name)                  -- same as
routeNext delayedRouter

  hoistServerWithContext _ ctx f handler = hoistServerWithContext
(Proxy @api) ctx f . handler
```

```
parseQueryParamFromRequest :: (FromHttpApiData a) => String ->
Request -> DelayedIO a
parseQueryParamFromRequest name req = do
  let params = queryString req
  case find ((== nameB) . fst) params of
    Just (_, Just rawValue) -> case parseQueryParam (decodeUtf8
rawValue) of
      Right parsed -> return parsed
      Left parseError -> delayedFail $ err400{errReasonPhrase =
"Failed to ?" <> name <> ":\n" <> T.unpack parseError}
    Just (_, Nothing) -> delayedFail $ err400{errReasonPhrase = "?"
<> name <> " requires a value"}
    _ -> delayedFail $ err400{errReasonPhrase = "?" <> name <>
"=<value> is required"}
 where
  nameB = encodeUtf8 $ T.pack name


instance (ToHttpApiData a, HasClient m api, KnownSymbol name) =>
HasClient m (MyQueryParam name a :> api) where
  type Client m (MyQueryParam name a :> api) = a -> Client m api
  clientWithRoute _ _ req value = clientWithRoute (Proxy @m) (Proxy
@api) $
    appendToQueryString (T.pack $ symbolVal (Proxy @name)) (Just
bytes) req
   where
    bytes = encodeUtf8 $ toQueryParam value
  hoistClientMonad _ _ f = (hoistClientMonad (Proxy @m) (Proxy
@api) f <$>)
```

Verb

HasServer (MyVerb name status contentType a)
HasClient m (MyVerb name status contentType a)
MimeRender contentType a
MimeUnrender contentType a

```
type MyPOSTAPI = MyVerb 'MyPOSTName 201
type MyGETAPI = MyVerb 'MyGETName 200

data MyVerb (name :: MyVerbName) (status :: Nat) contentType (a ::
Type)


instance (MimeRender contentType a, SingI name, KnownNat status) =>
HasServer (MyVerb name status contentType a) ctx where
  type ServerT (MyVerb name status contentType a) m = m a

  route ::
    forall env.
    Proxy (MyVerb name status contentType a) ->
    Context ctx ->
    Delayed env (Handler a) ->
    Router env
```

```
  route _ _ delayedRouter = StaticRouter M.empty [handleRequest]
   where
    method = case (sing @name) of
                                                -- step 1: compute
method name
      SMyGETName -> "GET"
      SMyPOSTName -> "POST"
    status = statusFromNat (Proxy @status)
    handleRequest :: env -> Request -> (RouteResult Response -> IO
ResponseReceived) -> IO ResponseReceived
    handleRequest env req k = runAction delayedRouter' env req k $
\a -> do
      Route $ responseLBS status [header] $ mimeRender (Proxy
@contentType) a                                  -- step 3:
compute response
     where
      header = (hAccept, renderHeader $ contentType (Proxy
@contentType))
      routeNext Delayed{..} =
        Delayed
          { methodD = methodD >> validateMethodName (requestMethod
req) method                                -- step 2: validate
method
          , ..
          }
      delayedRouter' = routeNext delayedRouter
      delayedRouter'' = delayedRouter `addMethodCheck`
validateMethodName (requestMethod req) method -- same

  hoistServerWithContext :: Proxy (MyVerb name status contentType
a) -> Proxy ctx -> (forall x. m x -> n x) -> m a -> n a
  hoistServerWithContext _ _ f = f


instance (MimeUnrender contentType a, SingI (name :: MyVerbName),
RunClient m) => HasClient m (MyVerb name status contentType a)
where
  type Client m (MyVerb name status contentType a) = m a
  clientWithRoute _ _ req = C.runRequestAcceptStatus Nothing req'
>>= parseResponse
   where
    parseResponse :: C.Response -> m a
    parseResponse res = case mimeUnrender (Proxy @contentType)
(C.responseBody res) of
      Right v -> return v
      Left message -> C.throwClientError $ DecodeFailure (T.pack
message) res
    req' :: C.Request
    req' =
      req
        { C.requestMethod = case (sing @name) of
            SMyGETName -> methodGet
            SMyPOSTName -> methodPost
```

```
        }
  hoistClientMonad _ _ f = f

validateMethodName :: Method -> Method -> DelayedIO ()
validateMethodName expected method =
  if expected == method
    then return ()
    else delayedFail $ err400{errReasonPhrase = "Unknown method" <>
show method}
```

e.g. API

```
type API =
  MyPath "api"
    :> ( MyPath "a" :> (MyGETAPI PlainText String :<|> MyPOSTAPI
PlainText ())
          :<|> (MyPath "b" :> MyCapture "name" Int :> MyGETAPI
PlainText Int)
          :<|> (MyPath "c" :> MyQueryParam "name" String :>
MyQueryParam "id" Int :> MyGETAPI PlainText String)
          :<|> (MyPath "d" :> MyEmpty)
       )

apiHandler :: Server API
apiHandler = apiA :<|> apiB :<|> apiC :<|> emptyHandler
 where
  apiA =
    return "a hello"
      :<|> do
        liftIO $ putStrLn "POST called"
        return ()
  apiB x = return (x * x)
  apiC name idNumber = do
    return $ "Received:" <> show (name, idNumber)
```

==========

```
This
is
code
Monaco 14
```